

CSA 1718 – Artificial Intelligence

Name : P. Yaswanth

Reg No : 192524167

EXPERIMENT : Pattern Matching Using Prolog

Aim

To write and execute a Prolog program to implement **pattern matching** and check whether two given patterns or terms are identical.

Objective

- To understand pattern matching in Prolog.
- To compare two terms.
- To observe how Prolog uses **unification** for matching.

Theory

Pattern matching is an important concept in Prolog. Prolog compares two terms and tries to make them identical using **unification**. If both terms can be matched successfully, the result is true; otherwise, it is false.

For example:

apple = apple → Match

apple = mango → No Match

Variables can also be used in pattern matching. A variable can be unified with a value if the matching is possible.

Algorithm

1. Start the program.
2. Define a predicate for matching two terms.
3. Compare the two terms using unification.
4. If the terms match, return true.
5. If they do not match, return false.
6. Test the program using suitable queries.
7. Stop.

Prolog Program

% Pattern Matching Program

match(X, X).

pattern_match(X, Y) :-

match(X, Y).

Queries and Output

Query 1:

?- pattern_match(apple, apple).

Output:

true.

Query 2:

?- pattern_match(hello, hello).

Output:

true.

Query 3:

?- pattern_match(apple, mango).

Output:

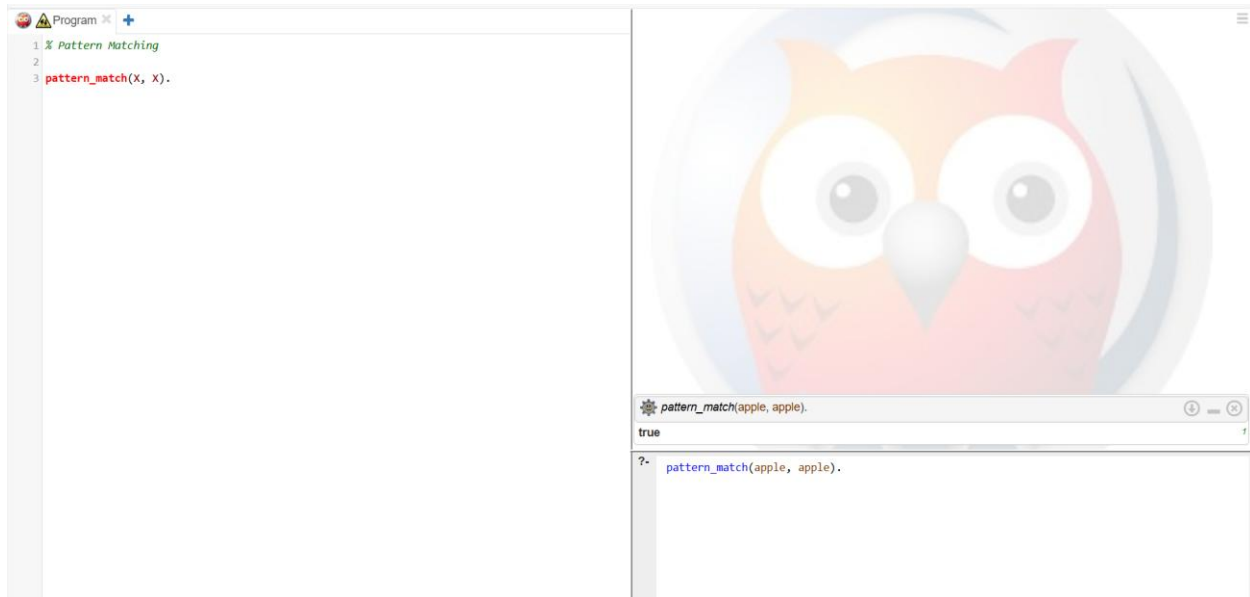
false.

Query 4:

?- pattern_match(10, 10).

Output:

true.



Explanation

- `match(X, X)` means both arguments must have the same value.
- `pattern_match(X, Y)` calls the match predicate.
- When both values are equal, Prolog successfully unifies them.
- When the values are different, unification fails.

Applications

Pattern matching is used in:

- Natural Language Processing
- Expert systems
- Knowledge representation
- Rule-based systems
- Artificial Intelligence
- Symbolic computation

Result

Thus, the **Prolog program for implementing pattern matching** was written and executed successfully.